Documentație Tehnică
Proiect POO ,,FixItNow"

Tudorache Robert-Cristian

11 ianuarie 2026

1 Arhitectura Aplicației

Aplicația este construită pe o arhitectură modulară, respectând principiile Programării Orientate pe Obiecte (OOP). Structura de clase este gândită pentru a separa logica de business (Service) de entitățile de date (Angajați, Electrocasnice).

Ierarhia Claselor

1. **Clasa Service (Singleton):** Este ,,creierul" aplicației. Gestionează listele de angajați și cereri. Fiind un Singleton, asigură o instanță unică accesibilă global, prevenind conflictele de date.
2. **Ierarhia Angajat:**
   - Clasa de bază abstractă *Angajat* definește atributele comune (nume, CNP, data angajării) și metoda pur virtuală `calculeazaSalariu()`.
   - Clasele derivate *Tehnician*, *Receptioner*, *Supervizor* implementează logica specifică de salarizare (coeficienți, bonusuri). *Tehnician* are, în plus, o listă de competențe și starea de disponibilitate.
3. **Ierarhia Electrocasnic:**
   - Clasa abstractă *Electrocasnic* definește aparatul generic.
   - Clasele derivate *Frigider*, *MasinaSpalat*, *Televizor* adaugă atribute specifice (congelator, capacitate, diagonala) și suprascriu metoda `afisare()`.
4. **ElectrocasnicFactory:** Implementează design pattern-ul Factory Method pentru crearea centralizată a obiectelor, decuplând logica de instanțiere de restul codului.
5. **Cerere:** Leagă un obiect Electrocasnic de un status (InAsteptare, Asignata, Finalizata) și de o complexitate a reparației.

Diagrama conceptuală a relațiilor

- Service ,,are" (agregare) → liste de Angajat și Cerere.
- Cerere ,,are" un Electrocasnic.
- Angajat (moștenește) → Tehnician, Receptioner, Supervizor.

2 Modul de Funcționare

Fluxul aplicației începe în `main.cpp`, care inițializează Singleton-ul Service și încarcă datele din fișierele CSV.

Procese Principale

1. **Importul Datelor:** Se utilizează funcții de parsare CSV robuste. Liniile sunt separate, iar datele sunt convertite. Erorile de formatare (ex: litere în loc de cifre) sunt prinse prin blocuri `try-catch`, asigurând continuitatea programului.
2. **Panoul de Control (Main Loop):** Utilizatorul interacționează printr-un meniu consolă cu trei secțiuni: HR (Angajați), Operațiuni (Cereri) și Raportare.
3. **Mecanismul de Load Balancing:** La opțiunea de asignare task-uri, Service-ul parcurge cererile în așteptare și caută tehnicianul compatibil cu cel mai mic număr de sarcini active, asigurând o distribuție echitabilă a muncii.
4. **Simularea Timpului:** Utilizatorul introduce un număr de minute simulate. Sistemul scade acest timp din durata rămasă a reparațiilor active. Când durata ajunge la 0, tehnicianul devine liber și primește bonusul financiar.

3 Elemente Tehnice și Modern C++

Proiectul utilizează facilități moderne ale limbajului C++ (Standard C++17) și biblioteci STL:

- **Design Patterns:**
  - *Singleton:* Folosit în clasa `Service` pentru acces centralizat.
  - *Factory:* Clasa `ElectrocasnicFactory` simplifică crearea obiectelor pe baza unui string (ex: „Frigider”), eliminând logica condițională repetitivă din `main`.
- **STL (Standard Template Library):**
  - `std::vector`: Pentru gestionarea dinamică a listelor de angajați și cereri.
  - `std::string` și `std::stringstream`: Pentru manipularea textelor și parsarea CSV.
  - `std::sort` cu *Lambda Expressions*: Utilizate în generarea rapoartelor (ex: sortarea angajaților după salariu) pentru un cod concis și lizibil.
- **Gestiunea Memoriei:** Utilizarea destructorilor pentru eliberarea memoriei alocate dinamic (`delete` în lanț) la închiderea aplicației.
- **Input Validation:** Utilizarea `cin.fail()`, `cin.clear()` și `cin.ignore()` pentru a preveni blocarea aplicației la introducerea de date eronate de la tastatură.

4 Descrierea Testelor

Testarea s-a realizat folosind fișiere externe (`tests/angajati.csv`, `tests/cereri.csv`) pentru a simula scenarii reale („Black Box Testing”).

Cazuri acoperite

- **Validare CNP:** S-a implementat algoritmul oficial de validare. Testele au inclus CNP-uri cu lungime greșită, caractere non-numerice și cifra de control incorectă. Toate au fost identificate corect.
- **Robustețe la Import:** Fișierele CSV au conținut linii alterate (prețuri scrise ca text, ani invalizi). Aplicația a raportat eroarea specifică (ex: `stod failed`) și a continuat încărcarea datelor valide.
- **Condiții de Funcționare:** S-a verificat condiția critică: aplicația refuză accesul la meniul de service dacă echipa nu este completă (min. 3 tehnicieni, 1 recepționer, 1 supervizor).